

SD: Instagram Clone

March 2026

Project realized by: **Pal Roland, Breban Vasile, Andrei Mandroc**

Contents

1	Introduction	2
1.1	Main Features	2
1.2	Main Database Tables	2
1.3	Main Views in the Frontend	3
2	Architecture	4
3	Tech Stack	6
4	Diagrams	7
4.1	Class Diagram	7
4.2	Package Diagram	8
4.3	Database Diagram	9
4.4	Use Case Diagram	10
4.5	Sequence Diagrams	11
4.5.1	Login Sequence Diagram	11
4.5.2	Create Post Sequence Diagram	12
4.5.3	Add Comment Sequence Diagram	13
5	Project Deliverable 2	15
5.1	Domain Model	15
5.2	Component Diagram	15
5.3	Deployment Diagram	16
6	Project Deliverable 3	18
6.1	Design Model Overview	18
6.2	Data Model Diagram	18
7	Conclusion	20

1 Introduction

This project is a simple Instagram Clone web application developed with Java Spring Boot for the backend and React for the frontend. It is designed as a social media platform where users can interact by creating and viewing content in a structured and user-friendly way. The application allows authenticated users to create, edit, and delete posts, add comments, use tags, upload pictures, and vote on posts or comments. Users can also browse content, view discussions, and interact with other users through the main features of the platform.

The project follows a clear client-server structure, where the frontend handles the user interface and the backend manages the business logic, security, and database operations. This approach makes the application easier to organize, maintain, and extend with new features in the future.

The application also includes an admin microservice. This microservice is separated from the main backend and is responsible for administrative actions, such as managing users and moderating content. In this way, the project respects a clean separation between the normal user functionality and the administrative functionality of the system.

1.1 Main Features

The main features of the Instagram Clone application are:

- User registration and login using authentication and JWT-based security.
- Creating, editing and deleting posts.
- Uploading pictures for posts and profiles.
- Adding tags to posts.
- Displaying posts in the feed.
- Adding, editing and deleting comments.
- Voting posts and comments using like and dislike actions.
- Displaying the vote score for posts and comments.
- Viewing user profiles and user posts.
- Administrative actions through a separate admin microservice.

1.2 Main Database Tables

The main database tables used in the application are:

- **users**
- **posts**
- **comments**
- **pictures**

- `tags`
- `post_tags`
- `post_votes`
- `comment_votes`
- `admin_action_logs`

These tables store the core data of the application. The users table stores account information, the posts and comments tables store the social content, while the vote tables store the like and dislike actions. The admin action logs table stores administrative operations performed through the admin microservice.

1.3 Main Views in the Frontend

The frontend interface includes the following main views:

- Login / Register page
- Home / Feed page
- Profile page
- Search page
- Notifications page
- Create Post modal
- Admin panel

These views allow users to navigate through the application and use its main functionalities in a simple way.

2 Architecture

The project follows a layered architecture on the backend and a separate frontend structure. This organization helps keep the code clear, modular, and easier to maintain. Each part of the application has a well-defined role, which makes the project easier to understand and extend.

- **Frontend** – The frontend is developed in React and is responsible for the user interface and user interaction. It sends requests to the backend, displays the received data, and allows users to use the main features of the application in a simple and interactive way.
- **Controller Layer** – The controller layer handles HTTP requests and returns responses to the client. It acts as the connection between the frontend and the backend logic, receiving input from the user and passing it to the appropriate service methods.
- **Service Layer** – The service layer contains the business logic of the application. It processes the application data, applies the main rules of the system, and coordinates the flow between controllers and repositories.
- **Repository Layer** – The repository layer manages database access and data persistence. It is responsible for saving, updating, deleting, and retrieving data from the PostgreSQL database through CRUD repositories.
- **Model Layer** – The model layer defines the main entities of the system, such as User, Post, Comment, Tag, Picture, and Vote. These classes describe the structure of the application data and the relationships between the main components of the system.
- **Configuration Layer** – The configuration layer contains security and general application configuration. It is used for settings related to authentication, authorization, Cloudinary configuration, and other technical aspects needed for the correct execution of the application.

The system also contains an admin microservice. This service communicates with the main backend when administrative operations are needed. The main backend focuses on the normal application features, while the admin microservice focuses on administrative actions.

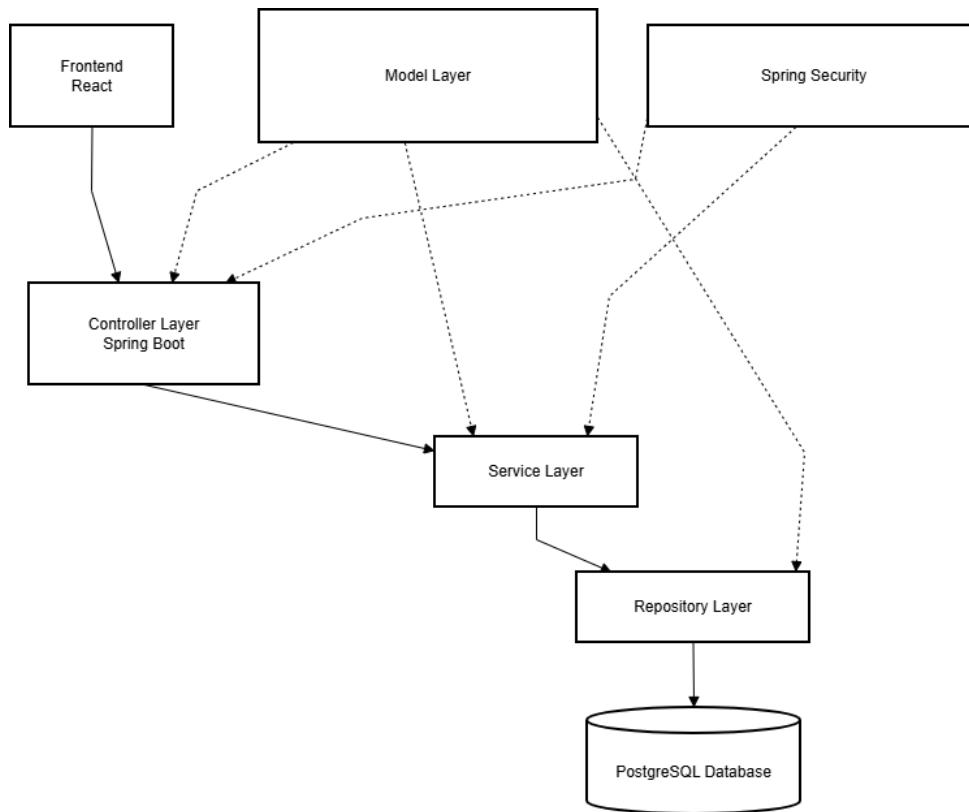


Figure 1: Architecture Diagram

The architecture diagram gives a general view of how the frontend, backend, admin microservice and database work together.

3 Tech Stack

The application is built using technologies for full-stack web development. The backend is implemented with Spring Boot, the frontend is implemented with React, and the database is managed using PostgreSQL.

- **Java** – main programming language used for the backend.
- **Spring Boot** – backend framework used to build REST APIs.
- **Spring Web** – used for REST API development.
- **Spring Data JPA** – used for database interaction.
- **Hibernate** – ORM framework used together with JPA.
- **Spring Security** – used for authentication and access protection.
- **JWT** – used for securing requests after login.
- **PostgreSQL** – relational database used for storing application data.
- **React** – frontend library used for building the user interface.
- **Cloudinary** – used for image upload and image storage.
- **JUnit** – used for unit testing.
- **Mockito** – used for mocking dependencies in tests.
- **Postman** – used for testing backend endpoints during development.
- **Maven** – dependency management and build tool.
- **Git and GitHub** – used for version control.

4 Diagrams

This section presents the main diagrams of the Instagram Clone project. The diagrams are used to describe the system from different perspectives: structure, packages, database design, user actions, runtime communication, deployment and data organization.

4.1 Class Diagram

The class diagram presents the main classes of the backend application and the relationships between them. It focuses on the domain entities used in the Instagram Clone, such as User, Post, Comment, Picture, Tag and Vote. The diagram also includes the main enumerations used by the system, such as Role, PostStatus and VoteType.

The User entity is connected to posts, comments and votes, because users can create content and interact with the content created by other users. Posts can have multiple comments, pictures, tags and votes. Comments can also receive votes. The admin microservice is represented through the AdminActionLog class, which stores administrative operations.

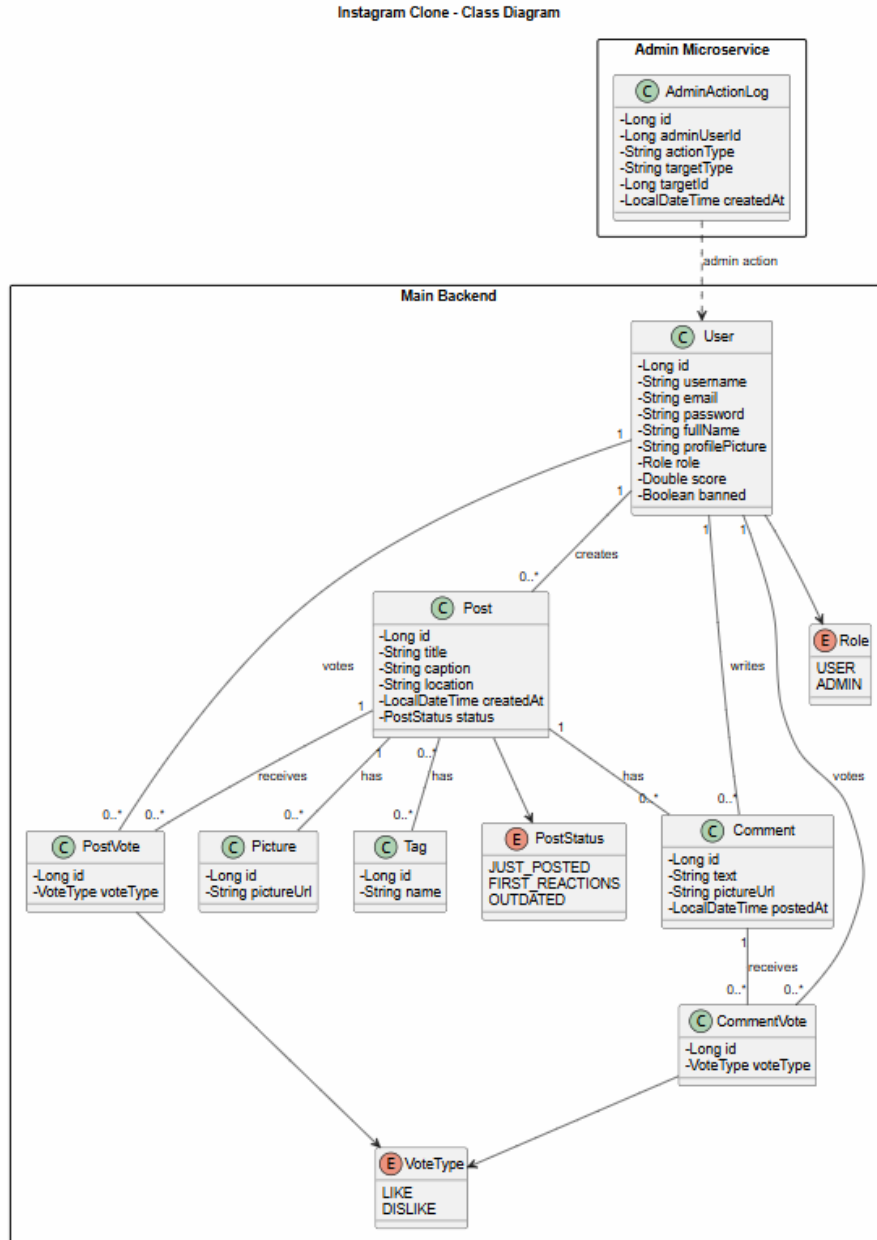


Figure 2: Class Diagram

The class diagram shows that the application is organized around users, posts, comments and voting actions.

4.2 Package Diagram

The package diagram shows the main code organization of the project. It presents the frontend, the main backend and the admin microservice as separate parts of the application. The backend packages follow the common layered structure: controller, service, repository, model and config.

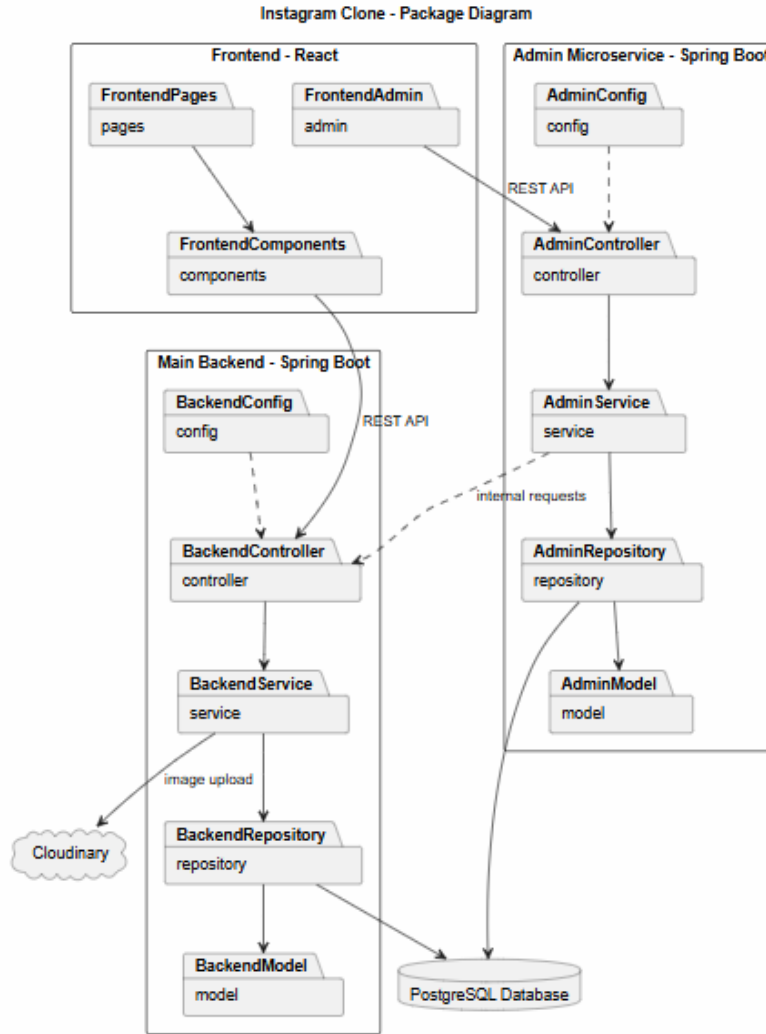


Figure 3: Package Diagram

The package diagram shows that the project is split into clear modules, which makes the application easier to maintain.

4.3 Database Diagram

The database diagram presents the main tables from the PostgreSQL database and the relationships between them. It is closer to the physical database structure than the class diagram, because it shows tables, primary keys and foreign keys. The users table is connected to posts, comments and votes, while posts are connected to comments, pictures, tags and post votes.

The many-to-many relationship between posts and tags is represented through the `post_tags` table. The voting system is represented through two separate tables: `post_votes` and `comment_votes`. This separation makes the vote logic clearer, because posts and comments are voted independently.

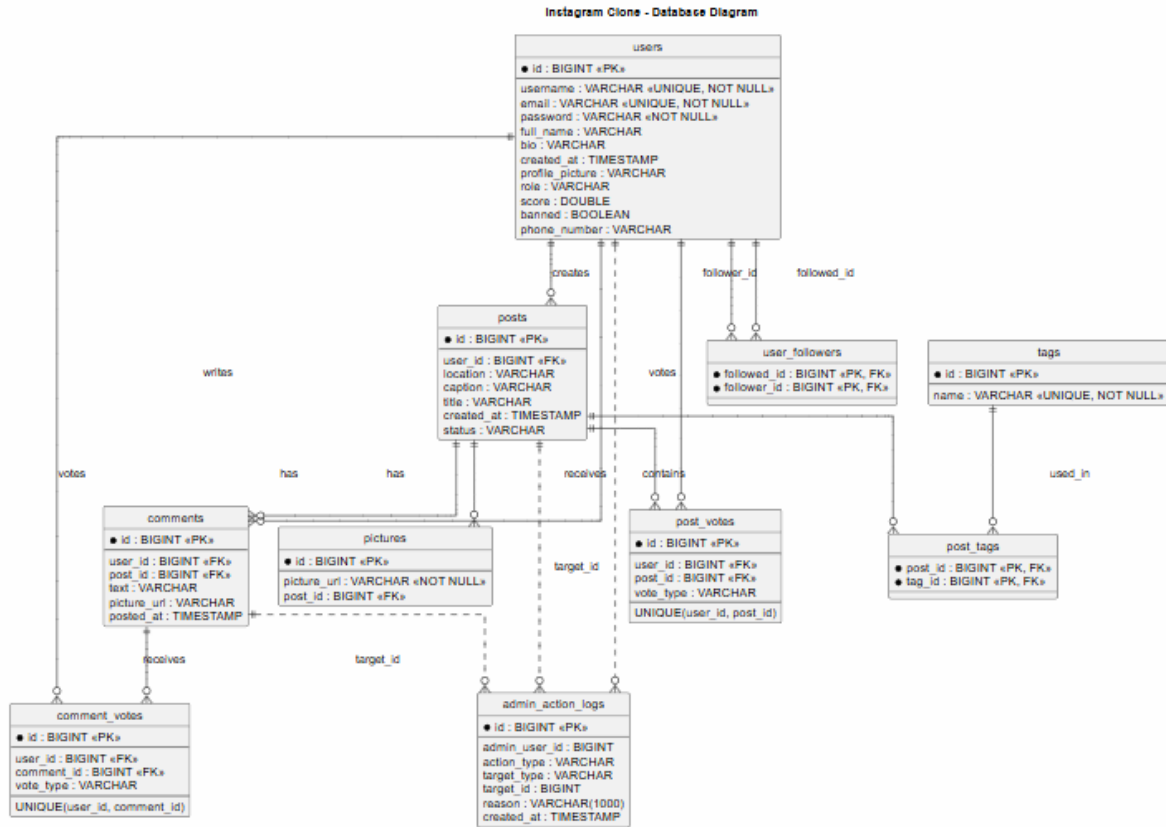


Figure 4: Database Diagram

The database diagram shows how the application data is stored and how the main tables are connected.

4.4 Use Case Diagram

The use case diagram presents the main actions that can be performed in the application. There are three main actors: Guest, User and Admin. A guest can register and log in, while an authenticated user can manage profile data, view the feed, create posts, add comments, vote and view notifications.

The Admin actor extends the normal user functionality and has access to administrative operations. The admin can manage users, moderate content and view admin logs through the admin panel.

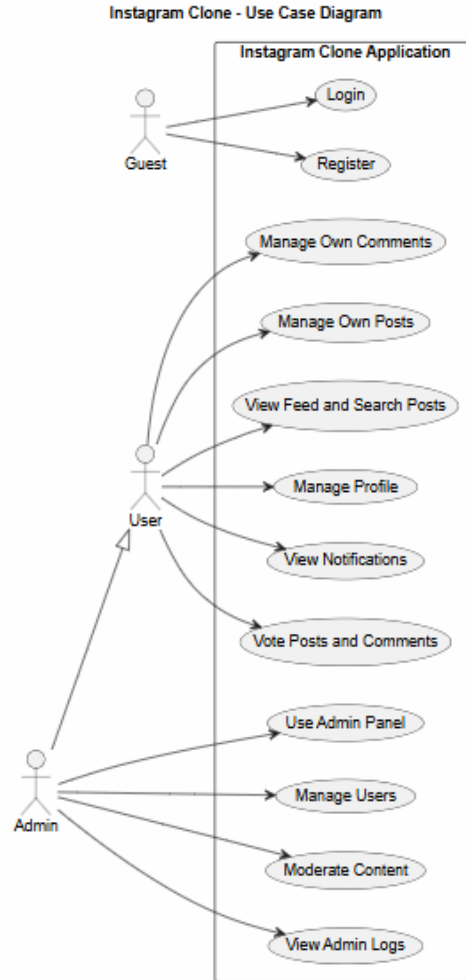


Figure 5: Use Case Diagram

This diagram gives a simple overview of the main user and admin actions supported by the application.

4.5 Sequence Diagrams

Sequence diagrams show how different parts of the system communicate during a specific action. In this project, the most important flows are login, creating a post and adding a comment. These flows are important because they connect the frontend, controllers, services, repositories and database.

4.5.1 Login Sequence Diagram

The login sequence diagram shows the authentication process. The user enters the login data in the React frontend, and the frontend sends the request to the backend. The backend verifies the user credentials, checks the encrypted password, verifies if the user is banned and then generates a JWT token if the login is valid.

The JWT token is returned to the frontend and used for future authenticated requests. This flow is important because most actions in the application require the user to be logged in.

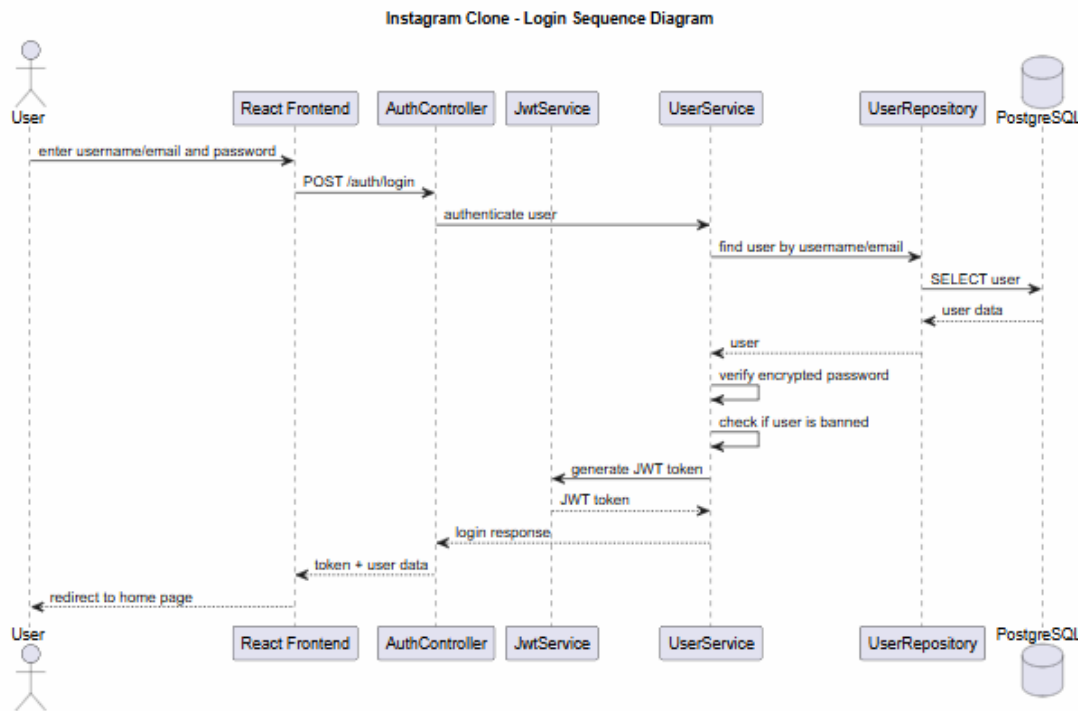


Figure 6: Login Sequence Diagram

The login flow shows how authentication is handled before the user can access the main application.

4.5.2 Create Post Sequence Diagram

The create post sequence diagram shows how a user creates a new post with an image and tags. First, the user selects an image and the frontend sends it to the upload endpoint. The backend uploads the image to Cloudinary and receives the image URL.

After the image URL is returned, the frontend sends the post data to the backend. The backend creates or finds the required tags, saves the post in the database and returns the created post to the frontend. The new post is then displayed in the feed.

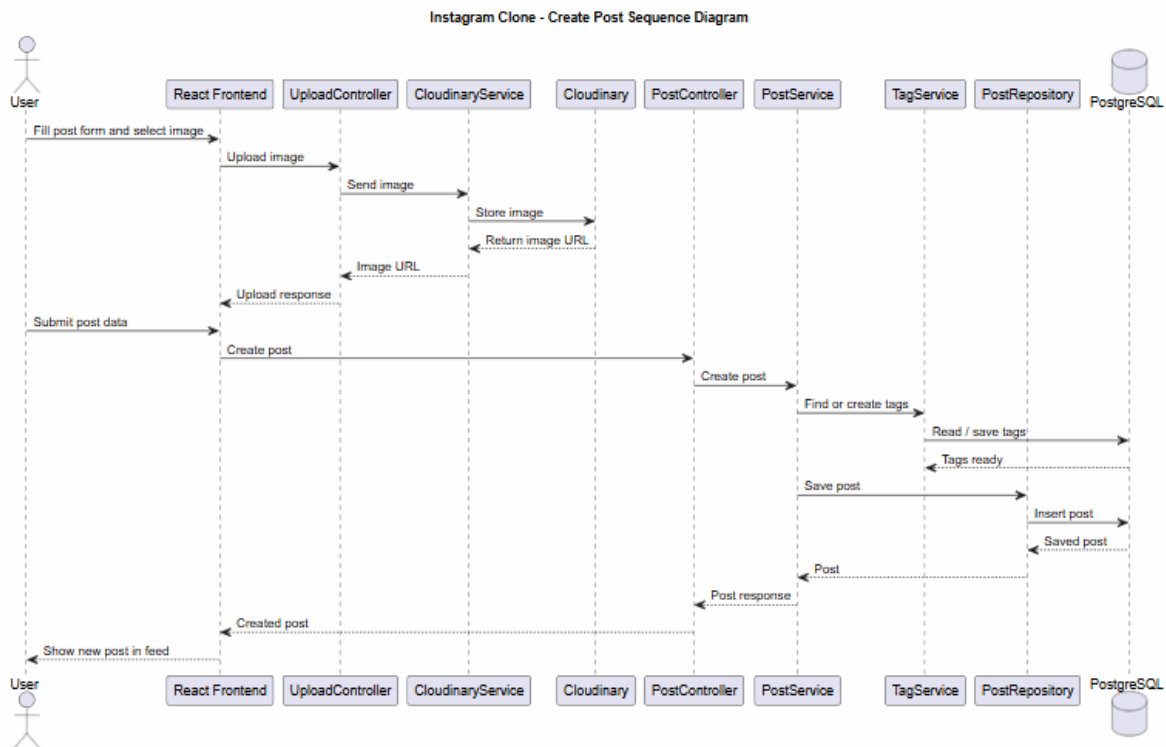


Figure 7: Create Post Sequence Diagram

This diagram shows how image upload and post creation are connected in the application.

4.5.3 Add Comment Sequence Diagram

The add comment sequence diagram presents the process of adding a comment to a post. The user writes a comment in the frontend, and the request is sent to the comment controller. The backend checks the related post, verifies if comments are allowed and then saves the comment.

After the comment is saved, the post status can be updated to show that the post has received its first reactions. This is useful because the application also manages the status of a post based on user interaction.

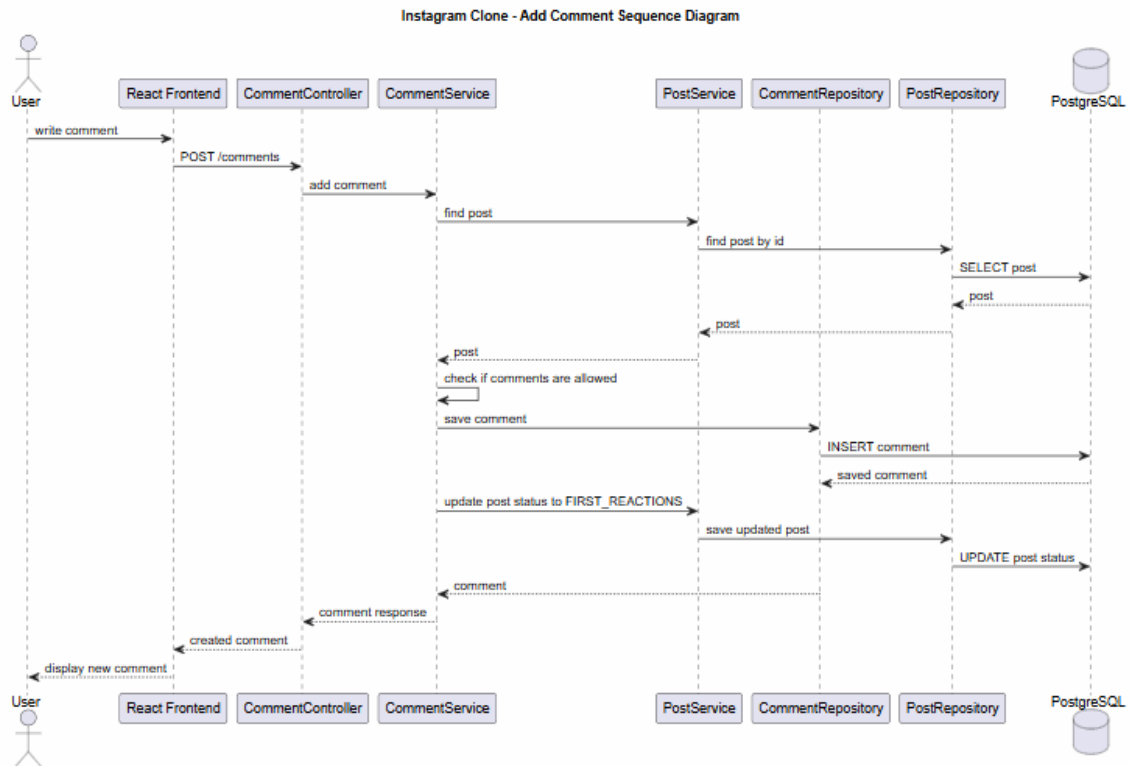


Figure 8: Add Comment Sequence Diagram

The comment flow shows how users interact with posts and how the backend updates the related data.

5 Project Deliverable 2

This deliverable focuses on the system structure from a higher-level perspective. It includes the domain model, the component diagram and the deployment diagram. These diagrams explain the main business concepts of the application, the communication between software components and the way the system is executed in practice.

5.1 Domain Model

The domain model presents the main concepts of the Instagram Clone application from a business perspective. It is less technical than the class diagram and focuses on what the application manages: users, profiles, posts, comments, pictures, tags, votes, notifications and admin actions.

In this model, a user can have a profile, create posts, write comments, give votes and receive notifications. Posts can contain pictures and tags, and they can also receive comments and votes. The admin action concept represents the administrative operations performed through the admin microservice.

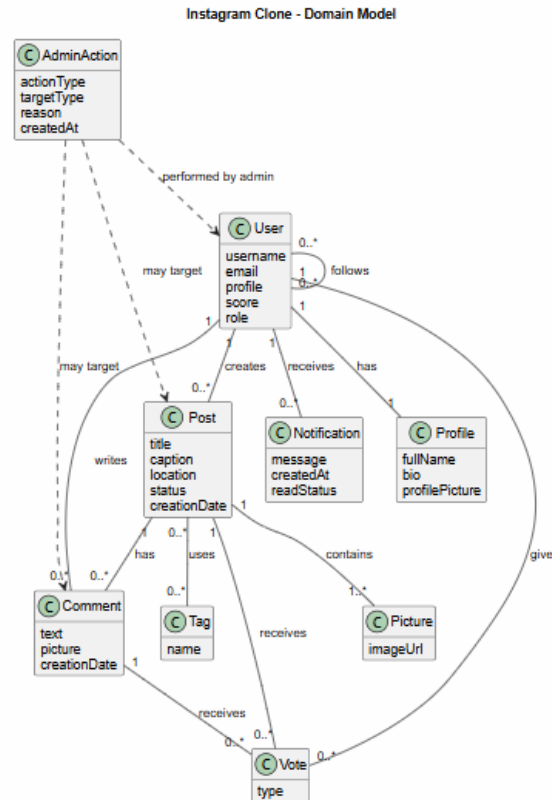


Figure 9: Domain Model Diagram

The domain model gives a simplified view of the main concepts managed by the application.

5.2 Component Diagram

The component diagram presents the main software components of the Instagram Clone application. The React frontend contains pages, reusable components and an admin

interface. The main backend contains controllers, services, repositories and security components. The admin microservice contains its own controller, service and repository.

The backend communicates with PostgreSQL for data persistence and with Cloudinary for image upload. The admin microservice can also communicate with the main backend when administrative actions must be applied to users, posts or comments.

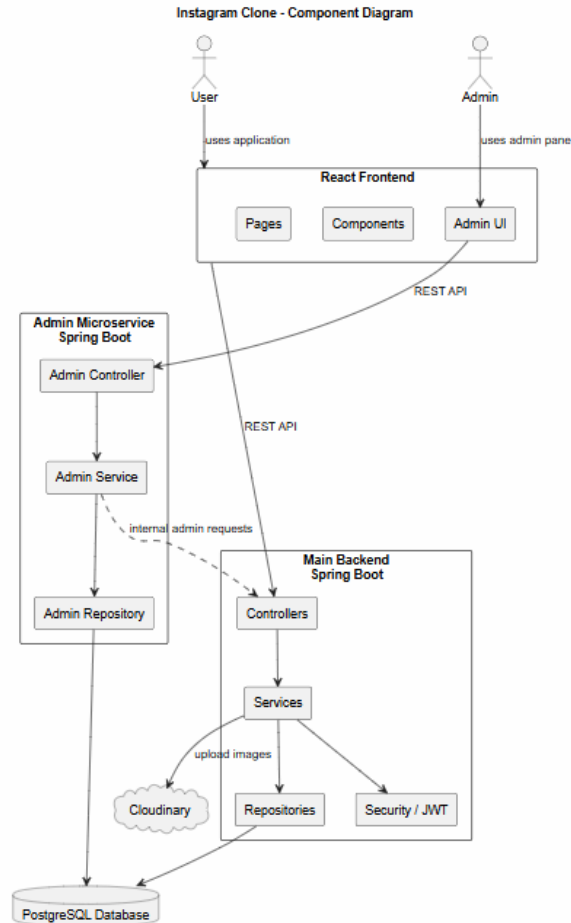


Figure 10: Component Diagram

The component diagram highlights the separation between the frontend, main backend, admin microservice, database and image storage.

5.3 Deployment Diagram

The deployment diagram shows how the application runs at runtime. The user accesses the application from a web browser. The browser communicates with the React frontend, while the frontend communicates with the main backend and the admin microservice through REST APIs.

The main backend runs on port 9090 and handles the normal application features. The admin microservice runs on port 9091 and handles administrative operations. Both backend applications use PostgreSQL for data storage, while the main backend also communicates with Cloudinary for image upload.

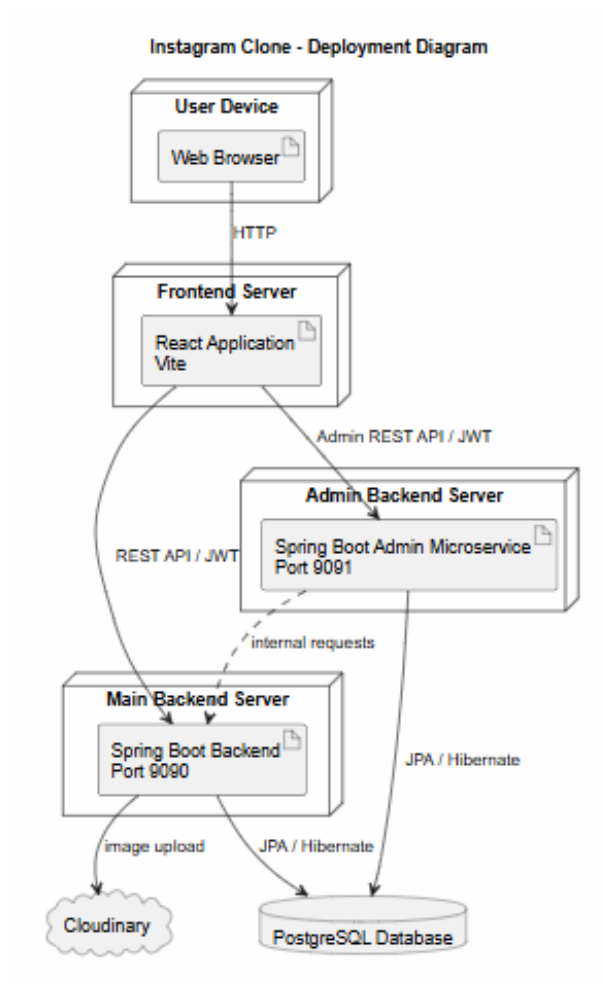


Figure 11: Deployment Diagram

The deployment diagram shows how the system components are deployed and connected during execution.

6 Project Deliverable 3

This deliverable presents the final design view of the application. It focuses on the design model and the data model. These two diagrams help explain how the application is organized internally and how the main data concepts are structured.

6.1 Design Model Overview

The design model overview presents the main layers of the system. The presentation layer contains the React pages, reusable components and the admin interface. The API layer contains the controllers that receive requests from the frontend. The business logic layer contains the services responsible for the main application rules.

The data access layer contains repositories, and the data layer contains PostgreSQL. Cloudinary is also included because image storage is an important part of the application. This design keeps the user interface, request handling, business logic and database access separated.

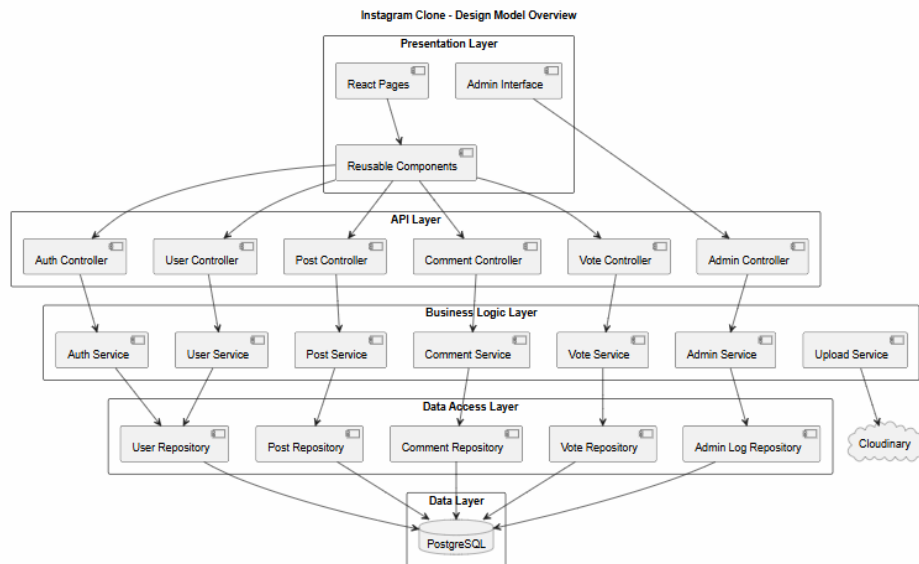


Figure 12: Design Model Overview

The design model shows the layered structure used to keep the application organized and easier to extend.

6.2 Data Model Diagram

The data model diagram presents the main data entities of the application and the relationships between them. It is more detailed than the domain model because it contains implementation-oriented elements such as identifiers, foreign keys and important stored attributes. At the same time, it is more conceptual than the database diagram, because it focuses on the application data and not only on the physical table structure.

The main data entities are User, Post, Comment, Picture, Tag, PostVote, CommentVote and AdminActionLog. Users create posts and comments, while posts contain pictures, tags, comments and votes. The vote entities store the like and dislike actions separately for posts and comments.

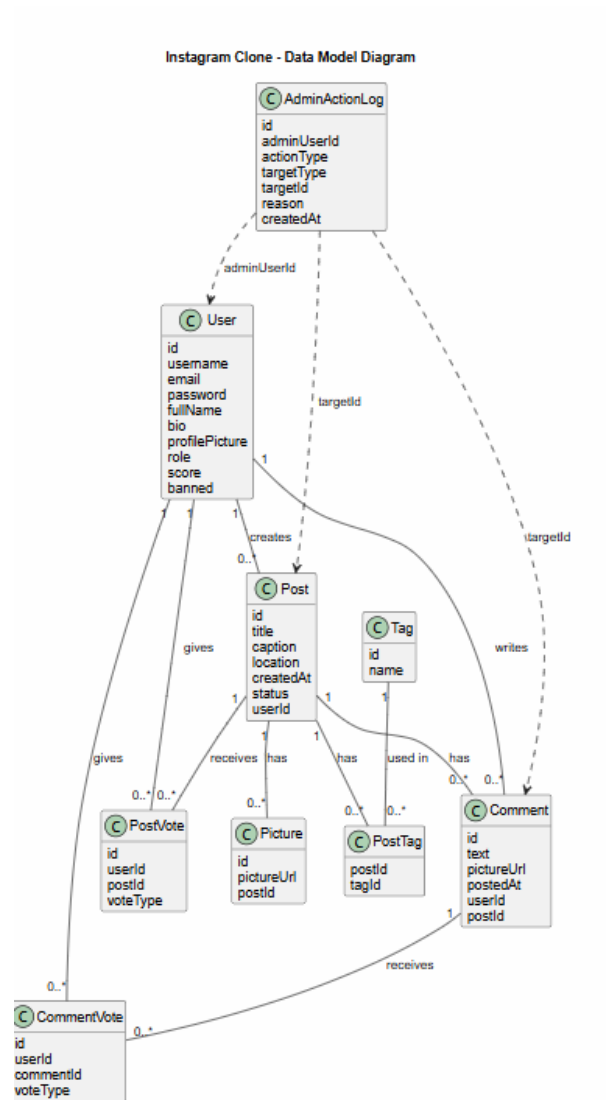


Figure 13: Data Model Diagram

The data model summarizes the most important data structures used by the Instagram Clone application.

7 Conclusion

The Instagram Clone project is a full-stack web application developed with React, Java Spring Boot and PostgreSQL. Its main purpose was to implement a simplified social media platform where authenticated users can create posts, upload images, add comments, use tags and interact with content through likes and dislikes. The project follows the main idea of Instagram, but it is adapted to the Software Design assignment requirements, with focus on clean architecture and modular code.

The frontend was implemented in React and contains the main pages and components of the application, such as the authentication pages, the feed, the profile page, the search page, the notification page, the post card component and the create post modal. It communicates with the backend through REST API calls and displays the data received from the server. This separation keeps the user interface independent from the backend logic.

The main backend was implemented using Java and Spring Boot. It follows a layered architecture based on controllers, services, repositories, models and configuration classes. The controllers handle HTTP requests, the services contain the business logic, and the repositories communicate with the PostgreSQL database. Features such as post creation, comment management, voting and profile operations are handled in separate service classes.

Security is handled using Spring Security and JWT authentication. Users must log in before performing protected actions, and after a successful login the backend generates a JWT token used by the frontend for authenticated requests. The project also uses Cloudinary for image handling, so uploaded images are stored externally and only their URLs are saved in the database.

The database was designed around the main entities of the application: users, posts, comments, pictures, tags and votes. Posts and tags have a many-to-many relationship, while posts and comments have separate voting tables. This allows the system to support multiple tags for each post and independent likes or dislikes for posts and comments.

The project also includes an admin microservice, separated from the main backend. The main backend handles normal user features, while the admin microservice focuses on moderation and administrative actions. This separation improves modularity and supports a microservice-based design.

The diagrams included in this documentation describe the project from different perspectives. They show the main entities, database structure, source code organization, user actions, important flows, components and deployment view of the system.

In conclusion, the Instagram Clone project demonstrates the development of a complete full-stack application using modern technologies. The use of React, Spring Boot, PostgreSQL, JWT and Cloudinary provides a practical implementation, while the layered architecture and the separation between frontend, backend, database and admin microservice make the project easier to maintain and extend.